

FIAP

Engenharia de Software — 3ESR — 2026

Atividade Avaliativa

Projeto Banco — API

Professor	Rafael S. Novo Pereira
Disciplina	Engenharia de Software (3ESR)
Entrega	Até 05/05/2026 (segunda) hora de término da aula
Formação	Individual, dupla ou trio Sem consulta ao celular.
Publicação	Portal FIAP
Pontuação	100 pontos

1. Contexto

Vocês foram contratados para construir o backend de um banco digital. O sistema precisa cadastrar clientes (que podem ser pessoa física ou jurídica), vinculá-los a uma agência, e permitir que cada cliente contrate um produto bancário.

Nem todo processamento de contratação pode ser feito de forma síncrona: análise de crédito, validação de convênio com empresa empregadora ou ativação de equipamento físico levam tempo, podem falhar e precisam ser reprocessadas.

2. Domínio (obrigatório)

A modelagem deve refletir o diagrama trabalhado em aula. Toda entidade marcada com asterisco (*) é obrigatória no modelo final.

Regras de modelagem

- **Cliente** é uma entidade abstrata com herança/discriminator para PessoaFisica (CPF, dataNascimento) e PessoaJuridica (CNPJ, razãoSocial).
- **Entidades obrigatórias:** Projeto Banco, Cliente, PF, PJ, Agência, Produto.
- **Produto** é uma classe-base com três especializações: MaquinaDeCartao, ReceberSalario, Emprestimo. Cada equipe escolhe um produto para implementar com regra de negócio real (a escolha vai no README).
- Um cliente pertence a uma única agência. Um cliente pode ter N contratações ao longo do tempo.

3. Escopo Técnico

3.1. Stack obrigatória

A stack é a mesma trabalhada em sala. Não é permitido substituir componentes do core.

Camada	Tecnologia
Runtime	.NET 8.0
API	ASP.NET Core Web API
ORM	Entity Framework Core
Banco	Oracle (oracle.fiap.com.br:1521/ORCL) — credenciais individuais (RM)

3.3. Endpoints mínimos

Método	Rota	Descrição
POST	/api/clientes/pf	Cadastra pessoa física
POST	/api/clientes/pj	Cadastra pessoa jurídica
GET	/api/clientes/{id}	Busca cliente por id
POST	/api/agencias	Cadastra agência
GET	/api/agencias/{id}	Busca agência
POST	/api/contratacoes	Solicita contratação (publica na fila)
GET	/api/contratacoes/{id}	Consulta status da contratação

3.4. Testes — fluxos críticos

A quantidade mínima de testes é livre, mas todos os fluxos críticos abaixo precisam estar cobertos. **Cobertura insuficiente derruba pontos da rubrica.**

Fluxos críticos que devem ter teste

- Cadastro de cliente PF e PJ (validação de CPF/CNPJ duplicado).
- Vincular cliente a agência inexistente (deve retornar 404 ou 400).
- Solicitação de contratação válida (deve publicar na fila e retornar 202/201).
- Solicitação de contratação para cliente inexistente (deve retornar 404).
- Consulta de status de contratação após processamento.
-

4. Entrega

4.1. O que entregar

Artefato	Onde	Obrigatório
Diagrama de classes UML	/docs/diagrama-classes.png (ou .drawio, .puml)	Sim
Código-fonte funcional	Repositório raiz	Sim
README.md	Raiz do repositório	Sim

4.2. README — conteúdo mínimo

O README deve conter, na ordem:

- **1. Identificação:** nome completo + RM de cada integrante (máx. 3).
- **2. Produto bancário escolhido** e justificativa.
- **3. Decisão de modelagem de filas** (1 fila? N filas? exchange?) com justificativa.
- **4. Diagrama de classes** (imagem incorporada).
- **5. Como rodar localmente:** pré-requisitos, comando do Docker para o RabbitMQ, dotnet ef database update, dotnet run.
- **6. Endpoints disponíveis** com exemplos de request/response em JSON.
- **7. Como executar os testes** (dotnet test) e print do resultado.
- **8. Print do painel do RabbitMQ** mostrando a fila com mensagens processadas.
- **9. Print da API rodando no Swagger** com pelo menos uma contratação aprovada.

4.3. Como entregar

- Plataforma da FIAP

Após o prazo, commits no main não serão considerados — a avaliação usa o último commit antes da deadline.

5. Rubrica de Avaliação (100 pontos)

Critério	Pontos	O que esperamos ver
Modelagem (UML + Domínio)	20	Diagrama de classes condizente com o código, herança PF/PJ correta, relacionamentos com cardinalidade, sem entidade órfã, no draw.io, mermaid não será aceito.
API REST funcional	40	Todos os endpoints da seção funcionando, retornos HTTP corretos, validação básica

Prints e explicação de uso de cada API e evidência de funcionamento	20	Evidências
Persistência (EF Core + Oracle)	10	Migrations aplicadas, Oracle conectando
README	10	README completo

Penalidades

- **Atraso na entrega: -10 pontos por dia**, limitado a 3 dias.
- **Cópia entre equipes (verificada por similaridade de código): nota zero** para todas as equipes envolvidas.

6. Diferenciais por tamanho da equipe

Para manter a justiça da avaliação, o escopo escala conforme o número de integrantes:

Tamanho	Escopo mínimo
Individual	Implementar 1 produto + Cliente (PF e PJ) + Agência + evidências e diagrama
Dupla	Tudo do individual + regra de negócio extra (ex.: cálculo de score, taxa MDR variável, validação de convênio)
Trio	Tudo da dupla + 2 tipos de produto implementados.

Não serão aceitas equipes maiores de 3 pessoas, uso de celular ou comunicadores proibido.

8. Dúvidas frequentes (antecipadas)

Posso usar outra linguagem além de C# .NET 8?

Não. A stack é fixa para que possamos avaliar todos no mesmo padrão visto em aula.

Posso usar Postgres ou SQL Server em vez de Oracle?

Não. Use Oracle FIAP — a credencial é a mesma do Sprint 3.

Preciso implementar os 3 produtos?

Não. Implemente 1 (individual/dupla) ou 2 (trio). Os outros podem aparecer no diagrama de classes mas não precisam estar na API.

O Consumer precisa estar em outro projeto?